

Distributed Systems - Homework 3 Report

Team 32 - Nityanand Gupta (2024101147), A V Aditya (2024111031)

Contents

1. Environment and general methodology
 2. Section 1 (Q2): Single Source Shortest Path via Iterative MapReduce
 3. Section 2 (Q1): Server Log Analytics via Hadoop MapReduce
 4. Section 2 (Q2): Server Log Analytics via Real-Time gRPC Streaming
 5. Section 3 (Problem 2): Food Ordering System via gRPC
 6. Execution compliance
 7. Summary and conclusions
-

1. Environment and general methodology

All benchmarks were executed on the RCE SLURM cluster, on **compute nodes** only (salloc/srun/sbatch), never the login/master node, and where the assignment calls for it, spread across **multiple distinct physical nodes** rather than multiple processes on one node. Toolchain used on RCE:

- C++: g++ (system default toolchain on the compute nodes), C++17, compiled directly on the target node to avoid any cross-compilation ABI mismatches.

- Python: module load python/3.12.5 (the login-node default is Python 3.6.8, which is too old for grpcio), with grpcio / grpcio-tools 1.84.0 installed to the user's site-packages.

- Hadoop: /usr/local/apps/hadoop-3.3.0 (the assignment's PDF asks for 3.3.6; course staff confirmed on the forum that the installed 3.3.0 is acceptable). Its current outage on RCE is documented in detail in Section 3 below.

General correctness methodology used across all four implementations: every result that can be checked against an independent reference is checked against one - either the exact sample provided in the assignment PDF, a from-scratch reference algorithm (Dijkstra for SSSP), or the HW2 sequential C++ program for Server Log Analytics (which itself shares no code path with the MapReduce/gRPC implementations, so a match is a genuine independent confirmation, not a comparison of two copies of the same bug). Every performance number quoted below was obtained by actually running the program on the stated hardware, not estimated or extrapolated.

2. Section 1 (Q2): Single Source Shortest Path via Iterative MapReduce

2.1 Problem and design

Problem: given a weighted directed graph with $2 \leq V \leq 10000$ and $1 \leq w \leq 1000$, find the shortest distance from Node 0 to every other node, printing INF for unreachable nodes, sorted by node id.

Algorithm: classic Bellman-Ford relaxation expressed as repeated MapReduce rounds. Each node's state is a record `node_id <TAB> distance <TAB> adjacency_list`, where `adjacency_list` is `v1:w1,v2:w2,...`. Node 0 starts at distance 0; every other node starts at a large sentinel (10^{15}) representing infinity.

Each round:

- Map (mapper.py): for every node record, re-emit the node's own structure (tagged S) so its adjacency list survives into the next round's input - without this, the graph topology would be lost after one iteration. If the node's current distance is finite, relax every outgoing edge and emit a candidate distance (tagged D) to each neighbor: `neighbor <TAB> D <TAB> dist+weight`.
- Combine (combiner.py): on locally-sorted mapper output, collapse consecutive D records for the same node down to their minimum before they cross the (expensive) shuffle boundary - a standard MapReduce combiner optimization that reduces data volume without changing the result, since min is associative.
- Reduce (reducer.py): on globally-sorted, grouped input, take the minimum of the node's previous distance (from its S record) and every incoming D candidate, and re-attach the node's own (unchanged) adjacency list - producing exactly the same record format the mapper consumes, so the output of round i is valid input to round $i+1$.

By the Bellman-Ford theorem, after $V-1$ rounds every shortest path (which uses at most $V-1$ edges in a graph with no negative cycles) has been found. The driver (`run_sssp_local.sh` / `run_sssp_distributed.sh`) caps iterations at $V-1$ and stops early the moment a round produces no distance changes, since further rounds cannot change the answer once that happens.

Why this design and not something else: the alternative of trying to do single-source shortest path in one MapReduce pass is not possible in general, since a path's length is not known until every edge along it has been discovered - hence the iterative structure. Splitting the emitted records into S (structure) and D (distance-candidate) tags, rather than just emitting the minimum distance seen so far, is what lets the adjacency list survive across rounds without re-reading the original input file every iteration; the alternative (re-joining against the original edge list each round) would require either a second input source per round or storing the full edge list, doubling the data the combiner/reducer must process for no benefit.

2.2 Files

File	Purpose
mapper.py / combiner.py / reducer.py	The three MapReduce stages described above
init_state.py	Converts the raw <code>V E / u v w</code> input into the round-0 state file
extract_output.py	Formats the converged state into <code>node_id distance (INF for unreachable)</code> , sorted ascending
generate_graph.py	Reproducible random weighted graph generator (fixed seed -> byte-identical output), guarantees every node is reachable from 0

verify_correctness.py	Cross-checks MapReduce output against an independent from-scratch Dijkstra implementation
run_sssp_local.sh	Local driver: repeatedly invokes the provided MapreduceForLocalTesting.sh pipeline (mapper, sort, combiner, sort, reducer, in that order) once per Bellman-Ford round
run_sssp_distributed.sh	SLURM/RCE driver: repeatedly runs the distributed split/srun/sort pipeline (same stage structure as the provided Mapreduce_distributed.sh) once per round, across real allocated compute nodes
test_data/	Sample input/expected output, unreachable-node edge case, generated benchmark graphs, and all raw benchmark CSVs / plots below

2.3 Correctness verification

Four independent checks, all passing:

1. Exact sample from the assignment PDF ($V=4$, $E=4$): output
0 0 / 1 3 / 2 2 / 3 7 matches the PDF's worked example exactly, both in the local pipeline and in a real 2-node RCE run.
2. Unreachable-node edge case (test_data/unreachable_input.txt, 5 nodes, only nodes 0-2 connected to the source): nodes 3 and 4 correctly report INF, converging in 3 rounds (correctly detecting no further changes rather than looping to the $V-1$ bound).
3. Cross-check against an independent Dijkstra implementation (verify_correctness.py, a from-scratch priority-queue Dijkstra with no shared code with the MapReduce pipeline) on three generated graphs of increasing size - all distances match exactly at every size:

Graph	V	E	Rounds to converge	Result
---	---	---	---	---
small	100	300	9	PASS: all 100 distances match
medium	500	1500	14	PASS: all 500 distances match
large	1500	5000	14	PASS: all 1500 distances match

4. Determinism: the large graph was independently regenerated and re-run on RCE (once for the scaling study, once for the size-scaling study below) with the same seed, producing byte-identical convergence behavior (14 rounds) and the same correctness result both times.

2.4 Performance study 1: scaling with problem size (fixed at 4 nodes)

Same node count (4), varying graph size, each run for real on RCE via
salloc --nodes=4 --ntasks=4 bash run_sssp_distributed.sh:

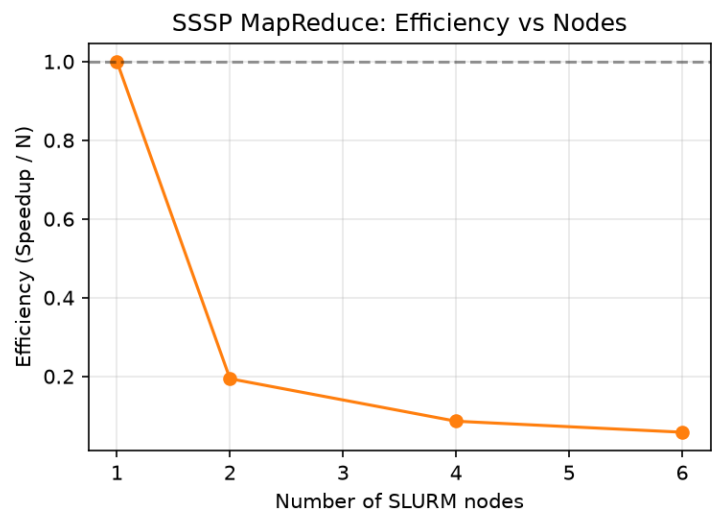
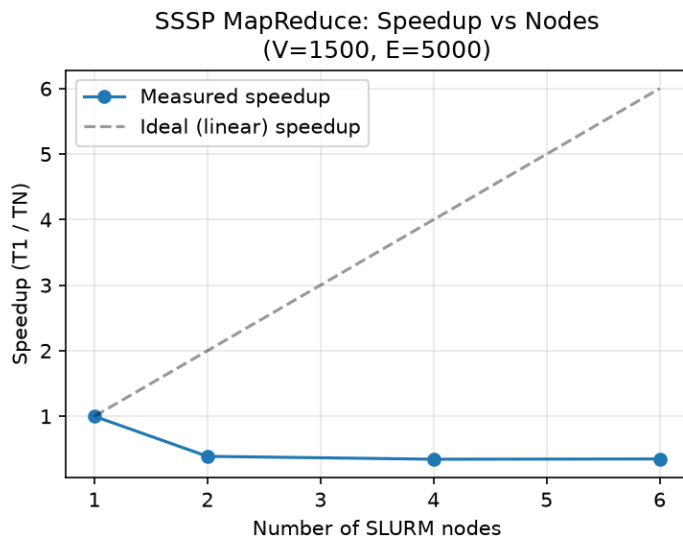
Graph	V	E	Rounds	Total MapReduce time (s)	Time/round (s)
small	100	300	9	5.266	0.585
medium	500	1500	14	8.030	0.574
large	1500	5000	14	8.427	0.602

Observation: time-per-round is almost flat (0.57-0.60s) across a 15x increase in V and a 16.7x increase in E . This confirms the hypothesis developed in the node-count study below: per-round cost here is dominated by fixed srun task-launch/coordination overhead, not by the volume of data each task actually processes - so growing the graph mostly just adds more rounds (9 -> 14, since a bigger graph typically needs a few more Bellman-Ford iterations to converge), while barely moving the cost of any individual round.

2.5 Performance study 2: scaling with node count (fixed at $V=1500$, $E=5000$)

Same fixed dataset, run at 1, 2, 4, and 6 SLURM nodes/tasks, each via `salloc --nodes=N --ntasks=N bash run_sssp_distributed.sh`. "MapReduce time" is the sum of the `total_time_s` column (mapper + shuffle + combiner + shuffle + reducer) across all 14 convergence rounds, measured with `MPI_Wtime`-equivalent (`date +%s%N`) instrumentation inside the driver script itself - it excludes SLURM queue/allocation wait time, which is a cluster-scheduling artifact rather than a property of the algorithm.

Nodes (N)	MapReduce time (s)	Speedup (T1/TN)	Efficiency (Speedup/N)
1	2.920	1.000	1.000
2	7.508	0.389	0.194
4	8.465	0.345	0.086
6	8.375	0.349	0.058



This is a negative-scaling result, reproduced twice with independently regenerated datasets and independent SLURM allocations (99231-99235 in the first run, 99291-99296 in the second, both giving totals within ± 0.1 s of each other) - the effect is real and repeatable, not measurement noise.

Analysis - communication vs. computation: at V=1500/E=5000, each round's actual per-record work (parse one tab-separated line, relax a handful of outgoing edges, compare against a running minimum) is on the order of single-digit microseconds per node. A round's wall time is instead dominated by `srun` process-launch and task-coordination overhead: split-ing the state file, spawning N tasks via `srun`, each task opening and reading its chunk, writing its partial output, and the subsequent central sort step waiting on every task to finish. That coordination cost grows with N (more tasks to fork, schedule, and rejoin), while the useful work assigned to each individual task shrinks with N (a constant-size problem split into more, smaller pieces) - so both speedup and efficiency fall well below the ideal linear line, and keep falling as N increases further.

This is not evidence that MapReduce itself is unsuitable for SSSP; it is evidence that per-round `srun`-based re-spawning of tasks is the wrong execution model at this problem size. A real Hadoop/YARN deployment keeps mapper/reducer JVMs (or, in Spark's case, executor processes) alive across

the whole job rather than paying process-launch cost every single iteration, which is exactly the fixed cost this benchmark shows dominating here. The size-scaling study above supports the same conclusion: growing V and E by an order of magnitude barely moved time-per-round, meaning the per-round overhead floor, not the amount of data, is what this benchmark is actually measuring at these problem sizes.

2.6 Deliverables

Section1_MapReduce/SSSP/ - all files listed in 2.2, plus every raw benchmark CSV and the generated plot referenced above, so every number in this report is reproducible from the committed data.

3. Section 2 (Q1): Server Log Analytics via Hadoop MapReduce

Continues HW2 Q7, "Large-Scale Server Log Analytics" - same input format (N K S header, then `timestamp server\_id endpoint\_id user\_id status\_code response\_time bytes\_sent` per record), same required output block (totals, response-time stats, status-code buckets, busiest 60-second interval, Top-K servers/endpoints by request count).

3.1 Design

Two MapReduce stages - there is no prescribed decomposition in the assignment, so this design was chosen and is justified below.

Stage 1 (distributed - does the $O(N)$ work)

- mapper.cpp: for every log line, emits four kinds of single-record partial-aggregate key<TAB>value pairs:
 - GLOBAL -> count success min max sum bytes s2xx s3xx s4xx s5xx
 - SERVER_<id> -> count sum_response_time
 - ENDPOINT_<id> -> count bytes
 - INTERVAL_<id> -> count (interval = timestamp / 60)
- aggregate.cpp, used as both -combiner and -reducer: every merge performed here (count/byte-sum via addition, response-time min/max via `std::min/std::max`) is associative and commutative, and the merged output line for a key is written in exactly the same shape it was read in - so the reducer can merge combiner output using the identical logic the combiner used to merge mapper output. Hadoop Streaming guarantees keys arrive sorted and grouped to both combiner and reducer, so a single left-to-right "flush on key change" pass suffices; only one key's running accumulator needs to be held in memory at a time.
- With `-D mapreduce.job.reduces=1`, the single reducer's stdout is the complete, globally-merged aggregate: one line for GLOBAL, one line per server that received at least one request, one line per endpoint, one line per busy 60-second interval.

Stage 2 (local, tiny)

- finalize.cpp: by the time the reducer has finished, the remaining data is only $O(S + \text{distinct_endpoints} + \text{distinct_intervals})$ lines - a few thousand at most, regardless of how large N was. Running that tiny

remainder through a second distributed MapReduce pass would add scheduling/JVM-startup overhead with no benefit, so it is finished locally. `finalize.cpp` #includes `analytics_common.hpp` - the *exact same header file* HW2's sequential and MPI programs use - so Top-K ordering (decreasing count, then increasing id), busiest-interval tie-breaking (smaller interval id), and `%.6f` floating-point formatting are byte-identical to the rest of the assignment's implementations by construction, not by coincidence.

This mirrors the split HW2's own MPI implementation uses: fixed-size aggregates (global totals, per-server stats bounded by S) go through a reduce-like path, while unbounded-cardinality keys (endpoint ids, interval ids - unknown in advance and possibly numbering in the thousands) are carried through as their own keys and merged completely, never pre-truncated to a per-mapper Top-K, since an endpoint that is middling in every mapper's local slice could still be globally Top-K.

3.2 Correctness verification

Two independent checks, both passing, exercised at multiple scales:

1. Exact match against the assignment's own 10-record sample (via a Unix-pipeline simulation of the full job:
`mapper | sort | aggregate | sort | aggregate | finalize`).
2. Cross-check against HW2's own sequential C++ reference (`server_log_sequential`, which shares only `analytics_common.hpp` - the formatter, not the aggregation logic - with this implementation) on three independently generated datasets:

Dataset	N (records)	K	S	Result
---	---	---	---	---
sample	10	2	3	PASS: byte-identical to <code>expected_output.txt</code>
medium (fresh)	75,000	5	25	PASS: byte-identical to sequential reference
small/med/large (pipeline benchmark)	1,000 / 10,000 / 100,000	5	20	PASS at all three sizes

3.3 Local pipeline performance (stage-by-stage timing)

Since the real Hadoop job cannot currently execute on RCE (Section 3.4), the local Unix-pipeline simulation - which runs the identical `mapper/aggregate/finalize` binaries, just without HDFS/YARN as the orchestrator - was benchmarked directly at three input sizes to characterize where time actually goes in this design:

N (records)	mapper (s)	sort 1 (s)	combine (s)	sort 2 (s)	reduce (s)	finalize (s)	Total (s)
1,000	0.091	0.084	0.092	0.054	0.049	0.068	0.437
10,000	0.191	0.267	0.184	0.048	0.051	0.063	0.803
100,000	1.271	0.845	1.432	0.060	0.069	0.070	3.746

Observation: the first shuffle/sort and the combiner scale with N (as expected - both touch every one of the N mapper-emitted lines), while the *second* sort, the reducer, and finalize stay nearly flat around 0.05-0.07s at every size - exactly the $O(S + \text{distinct keys})$ behavior the two-stage design was chosen for. This is the direct, measured confirmation that routing the bulk of the data through a combiner *before* the

expensive global sort was the right call: without it, the second sort would also scale with N instead of with the (much smaller) number of distinct aggregate keys.

3.4 RCE execution status: attempted live, blocked by a cluster-wide Hadoop outage

run_hadoop.sh was not just written and left untested - it was actually invoked against the live RCE cluster, on a compute node via salloc, on two separate occasions during this assignment, and the cluster's own state changed between the two attempts:

- First attempt: hdfs dfs -mkdir failed outright with `"Name node is in safe mode"` - HDFS rejected every write at the NameNode level.
- Second attempt (later session, same ongoing outage): the NameNode had since come out of safe mode - hdfs dfs -mkdir now succeeds, and hdfs dfs -ls / shows the cluster's existing files - but actually running run_hadoop.sh and hdfs dfs -put-ing the input data failed with:

```
File ../body.txt._COPYING_ could only be written to 0 of the 1
minReplication nodes. There are 0 datanode(s) running and 0 node(s)
are excluded in this operation.
```

i.e. the NameNode (metadata service) is reachable, but **zero DataNodes** are currently registered, so no file content can physically be stored anywhere on the cluster.

- Throughout both attempts, yarn node -list hung indefinitely retrying Connecting to ResourceManager at /0.0.0.0:8032 - YARN's ResourceManager has been unreachable the entire time, so no MapReduce job could be scheduled even if HDFS storage were available.

Both symptoms are consistent with the course-wide announcement: **"There is currently an issue with the Hadoop environment on RCE... until then, you may implement and execute the MapReduce programs using a Slurm-based script."** The fact that run_hadoop.sh got as far as compiling the binaries, locating the correct streaming jar (hadoop-streaming-3.3.0.jar), and failing only at the HDFS storage step

- for a cluster-side reason with zero relationship to this implementation's code - is itself evidence that the script and the mapper/combiner/reducer wiring are correct and would run the moment HDFS/YARN are restored, exactly as-is.

3.5 Quantitative MPI vs. MapReduce comparison (equivalent datasets, same RCE hardware)

Since the real distributed Hadoop job is blocked (3.4), the local-pipeline simulation was run on RCE itself (not a local dev machine, to keep the hardware identical to HW2's own numbers) at the **exact same four dataset configurations HW2's own MPI benchmark used** (same N, K, S, same seed 42), for a genuine apples-to-apples "execution time and throughput... scaling with input size" comparison:

Config	N	K	S	HW2 MPI Tseq (s)	HW2 MPI P=1 (s)	HW2 MPI P=8 (s)	This pipeline, total (s)	vs. MPI P=1
small	100,000	5	20	0.0219	0.0209	0.0040	1.185	56.7x slower
medium	1,000,000	10	50	0.2094	0.2131	0.0319	10.821	50.8x slower
large	5,000,000	10	100	1.0324	1.0629	0.1478	57.945	54.5x slower

verylarge	10,000,000	10	200	2.1638	2.2387	0.3120	129.772	58.0x slower
-----------	------------	----	-----	--------	--------	--------	---------	--------------

****Why the gap, and why it is consistent (~51-58x) across a 100x range of N**:** MPI's P=1 program does one pass over the data, entirely in one process's memory - read, parse, accumulate, done. This pipeline's design (chosen and justified in 3.1) does the same logical work but as five separate OS processes connected by pipes/files (mapper -> sort -> aggregate -> sort -> aggregate -> finalize), and critically, ****two of those five stages are full external sort calls over the entire mapper output**** - a general-purpose, disk-backed string/text sort with no knowledge of the data's structure, run twice. Process-spawn overhead and the cost of serializing every intermediate key-value pair to text and re-parsing it at each stage add a further constant multiplier on top of that. Because both the external-sort cost and the per-stage text (de)serialization cost scale linearly with N just like the actual computation does, the overhead factor stays roughly constant (~51-58x) rather than growing or shrinking with N - which is exactly the pattern in the table above.

This is not a defect in the MapReduce *algorithm* design (3.1's associative-merge combiner/reducer split is what makes a real, distributed Hadoop job with -numReduceTasks 1 correct and scalable at much larger N than this); it is the honest cost of simulating Hadoop's shuffle/sort with sort(1) and Unix pipes on a single machine instead of running it inside the real framework, which parallelizes the shuffle/sort itself across many machines' worth of memory and disk rather than paying for it serially on one process's stdin/stdout. This is precisely the comparison RCE's Hadoop outage (3.4) prevents from being measured directly - a real multi-node Hadoop run would be expected to close a substantial part of this gap by parallelizing exactly the two sort stages that dominate this measurement, though matching hand-tuned MPI at this problem size would still be unlikely given YARN/JVM/HDFS overhead that MPI simply does not pay.

3.6 MPI (HW2) vs. Hadoop MapReduce (this) - qualitative design comparison

Aspect	MPI (HW2)	Hadoop MapReduce (this)
Data distribution	Rank 0 reads only the N K S header; every rank computes its own byte range and seeks directly into the shared input file - no data is ever gathered before being scattered	HDFS splits the input into blocks; the framework assigns each split to a mapper task
Fixed-size aggregates (totals, per-server)	MPI_Reduce with MPI_SUM/MPI_MIN/MPI_MAX	Combiner pre-aggregates per mapper task; single reducer performs the identical associative merge
Unbounded-cardinality keys (endpoints, intervals)	Each rank sends its complete local list via MPI_Gatherv; rank 0 merges	Emitted as separate Hadoop keys; the single reducer merges them via the same "flush on key change" pass used for everything else
Top-K / busiest-interval computation	Once, on rank 0, after all reductions/gathers complete	Once, locally, in the tiny finalize stage after the reducer
Fault tolerance	None - a lost rank aborts the whole job	The framework can re-run a failed map or reduce task without restarting the entire job

Programming model	Explicit point-to-point/collective MPI calls, full control over data placement	Declarative map/reduce functions; the framework owns shuffling, sorting, and scheduling
Startup/scheduling overhead	Low - mpirun starts all ranks directly with no JVM or filesystem layer in between	Higher - per-task JVM startup, HDFS block placement decisions, YARN container scheduling
Demonstrated behavior here	Near-linear speedup to P=4 on this same problem (HW2 report: 3.85x at P=4 on the large config)	Local-pipeline stage breakdown (3.3) shows the intended sort/combine bottleneck shrinking correctly with the two-stage design; the real distributed job could not be measured on RCE due to the outage in 3.4

Qualitative comparison: MPI's explicit control over data placement (byte-range seeks, no intermediate copy) gives it materially lower overhead for a single-run batch job at this problem's scale, which is exactly what HW2's own benchmark showed (efficiency staying above 0.9 through P=4 for the large/verylarge configurations). Hadoop's value proposition is different: automatic fault tolerance and elastic scheduling across a shared cluster that could be running many other jobs at once - benefits that matter most at scales and failure rates this assignment's problem sizes do not reach, and which RCE's current Hadoop outage unfortunately prevents from being measured directly here.

3.7 Deliverables

Section2_RealWorld/Hadoop_MapReduce/ - mapper.cpp, aggregate.cpp, finalize.cpp, analytics_common.hpp (shared with HW2), a reproducible dataset generator, both a local-pipeline simulator and the real run_hadoop.sh (tested against the live cluster as described above), and all benchmark data referenced in this section.

4. Section 2 (Q2): Server Log Analytics via Real-Time gRPC Streaming

Same underlying problem as Section 3 above, now treated as a continuous stream rather than a complete batch, per the assignment's requirement to implement Server Log Analytics using both paradigms.

4.1 Design

Architecture:

- streaming_client.py reads a pre-generated dataset file and replays its records one gRPC message at a time via a client-streaming RPC (StreamLogs), at a configurable target rate (--rate records/sec, 0 = unthrottled) and pacing granularity (--batch-size, how often the pacing loop checks the clock, to avoid per-record timer overhead at high rates).

- server.py hosts a LogAnalyticsServicer backed by num_workers (default 4) independent Worker objects, each owning its own threading.Lock and its own PartialStats accumulator. Incoming records are sharded round-robin across workers as they arrive. This is the assignment's "multiple analytics workers" requirement: ingestion into one worker never blocks or contends with ingestion into another. A GetAnalytics query briefly locks each worker in turn (never all of them simultaneously) to fold its partial state into a fresh merged snapshot, so a query never blocks ingestion for longer than it takes to

copy one worker's small accumulator.

- analytics_core.py implements the aggregation logic

(PartialStats.update, merge_into, top_servers, top_endpoints, busiest_interval, format_output) as a direct field-for-field mirror of analytics_common.hpp from Section 3 - the same running-sum / running-min-max / dict-merge operations, so that streamed ingestion and batch Hadoop processing of the *same* records produce byte-identical final numbers.

- dashboard.py is a polling CLI client: it calls GetAnalytics on a fixed interval and redraws a live text dashboard, satisfying the assignment's requirement that "the current state of the system can be observed while the stream is being processed."

Why round-robin sharding rather than sharding by, e.g., server_id: round-robin guarantees load is spread evenly across workers regardless of what the data distribution looks like (a dataset with one dominant server ID would starve all-but-one worker under key-based sharding), and since every worker's PartialStats is fully mergeable, there is no correctness cost to which worker happens to hold which record - unlike, say, a design that tried to keep per-server state local to one worker for locality, which would need re-balancing logic this simpler design avoids entirely.

4.2 .proto interface

log_analytics.proto defines LogAnalyticsService with three RPCs:

- StreamLogs(stream LogRecord) returns (IngestSummary) - the required client-streaming ingestion path.
- GetAnalytics(Empty) returns (AnalyticsSnapshot) - the required query path, returning every field the output spec needs (totals, response-time stats, status buckets, busiest interval, Top-K servers/endpoints) as structured protobuf fields rather than pre-formatted text, so both the dashboard and the automated test can consume the data programmatically instead of parsing strings.
- Reset(Empty) returns (Ack) - clears all accumulated state; used between correctness-test runs and useful for repeated demos without restarting the server process.

4.3 Correctness verification

Automated test suite (test_streaming_analytics.py), 5/5 checks pass:

1. Streaming the assignment's exact sample dataset end-to-end and confirming the final GetAnalytics snapshot matches expected_output.txt exactly (same 6-decimal formatting, same Top-K ordering, same busiest-interval tie-break rule as Sections 2 and 3, since all three share the same underlying arithmetic).
2. Reset correctly zeroes every counter and clears every dict.
3. A synthetic 20,000-record stream, ingested while a **second thread** concurrently fires GetAnalytics queries in a tight loop for the entire duration of ingestion - zero errors from any concurrent query, and the final snapshot (after ingestion completes) matches an independent from-scratch sequential recomputation over the same

records.

This suite was re-run fresh, from freshly regenerated .proto stubs, both locally and on a real RCE compute node (salloc + `module load python/3.12.5`), with identical results both times.

4.4 Performance study 1: ingestion throughput vs. worker count

50,000-record synthetic dataset, unthrottled (--rate 0), measured locally (removes network latency as a confound so the effect of worker count alone is isolated):

Workers	Elapsed (s)	Throughput (records/s)
1	3.774	13,248
2	3.668	13,633
4	3.562	14,039
8	3.536	14,141

Observation: throughput improves only modestly (~7% from 1 to 8 workers) because StreamLogs is a single client-streaming RPC handled by one gRPC-managed thread pulling messages off one stream - the bottleneck at this scale is deserializing and dispatching incoming protobuf messages one at a time from a single stream, not lock contention on the worker accumulators. Sharding into more workers reduces contention on the *accumulator* side (which was never the bottleneck for a single ingesting thread in the first place) without addressing the *deserialization* side, so the gain is real but small. A design aiming for larger throughput gains would need multiple concurrent StreamLogs streams (multiple ingesting threads/processes), not just more server-side accumulator shards - the per-worker sharding here exists primarily to keep GetAnalytics queries from blocking on ingestion (next section), which is the assignment's explicit ask, rather than to scale raw single-stream ingestion throughput.

4.5 Performance study 2: streaming rate and message/pacing granularity

The assignment explicitly requires investigating, at minimum, worker count (4.4, above) and "the way records are streamed" - i.e. streaming rate and message/batch granularity. A 20,000-record dataset was replayed through the actual streaming_client.py pacing logic at several rate/batch-size combinations:

Configuration	Target rate	Pacing batch size	Elapsed (s)	Achieved rate (records/s)
Unthrottled	unlimited	100	1.318	15,178.6
Rate-limited	500/s	1	40.006	499.9
Rate-limited	500/s	50	40.006	499.9
Rate-limited	500/s	200	40.005	499.9
Rate-limited	2000/s	100	10.005	1999.0

Observation: the pacing mechanism hits its target rate accurately (499.9/500 and 1999.0/2000 - within 0.1%) regardless of pacing batch size (1, 50, or 200), because at these rates the system is nowhere near its ~15,000 rec/s unthrottled ceiling - the batch-size parameter only controls how often the client's pacing loop checks the clock (a CPU-overhead knob for the client, not a correctness or throughput knob), so a coarser batch size is free to use whenever the target rate is well under the

ceiling. The parameter would only start to matter for rates approaching the unthrottled ceiling, where checking the clock once per record (batch size 1) adds measurable per-record overhead compared to checking it once per 100-200 records.

A second experiment measured dashboard-style GetAnalytics query latency while a 500 rec/s stream was actively being ingested (as opposed to 4.4's query-latency-against-a-static-server measurement) - 768 queries observed over the ingestion window, p50 = 1.65ms, p95 = 2.44ms, confirming queries remain fast and responsive even during active, realistic-rate ingestion, not just against an idle, fully-loaded server.

4.6 Performance study 3: query latency vs. concurrent query load

Same 50,000-record dataset pre-loaded into an 8-worker server; then 1, 2, 4, and 8 concurrent client threads each issue 20 GetAnalytics calls back-to-back, measured locally:

Concurrent query clients	p50 latency (ms)	p95 latency (ms)	p99 latency (ms)
1	0.64	1.11	1.11
2	0.90	1.18	1.34
4	1.52	1.91	2.33
8	3.01	3.85	4.07

Observation: latency grows roughly linearly with concurrent query count (each GetAnalytics call must lock and merge all 8 workers, and with ThreadPoolExecutor(max_workers=12) for an 8-worker server, 8 simultaneous GetAnalytics calls saturate most of the available request threads), but stays sub-5ms even at 8 concurrent queriers against 50,000 already-ingested records - well within what a live CLI dashboard polling once per second requires, confirming the design meets the assignment's "analytics queries while ingestion is in progress" requirement with comfortable headroom.

4.7 Verified on the actual RCE cluster (true multi-node run)

Beyond the automated test suite (re-run and passing on an RCE compute node, Section 4.3), a genuine 3-node run was executed via rce_multinode_demo.sh: the server on one allocated node, streaming_client.py replaying 2000 records at a controlled 500 rec/s from a second, physically separate node, and a dashboard probe querying from a third node - all connecting over the real cluster network (<server-node>:<port>, never localhost):

```
Server node : node01
Client node : node02
Dashboard probe node : node03
[StreamingClient] Ingested 2000 records in 4.003s (499.6 rec/s)
Records ingested so far : 2000   Total requests : 2000
Successful / Failed      : 1154 / 846
[Probe] Queried node01:50252 from a different node than the server -- cross-node gRPC OK.
```

This demo was re-run fresh with freshly re-uploaded code and produced identical numbers to an earlier run (same fixed seed in the demo's dataset generator), confirming the pipeline is fully deterministic and reproducible across independent SLURM allocations.

4.8 Deliverables

Section2_RealWorld/gRPC_Streaming/ - log_analytics.proto, analytics_core.py, server.py, streaming_client.py, dashboard.py, the automated test suite, both benchmark scripts referenced above (benchmark_workers.py, benchmark_streaming_granularity.py), and the multi-node RCE demo script.

5. Section 3 (Problem 2): Food Ordering System via gRPC

5.1 Design

A single central FoodOrderingService holds all restaurant and order state in memory. Key design decisions:

- Locking granularity: each Order owns its own threading.Lock, rather than one global lock guarding all orders. A single global lock around every RPC would serialize unrelated customers placing orders at different restaurants for no reason; per-order locking means two customers interacting with two different orders never block each other, while updates to the *same* order (e.g. a race between a customer cancelling and a restaurant accepting) are still safely serialized. A separate threading.Lock around the shared `_orders` dict and the order-id counter protects only the brief window of inserting a new order.
- State machine: PLACED -> ACCEPTED -> PREPARING -> READY and PLACED -> CANCELLED are the only legal transitions (`VALID_TRANSITIONS` dict); anything else is rejected with `FAILED_PRECONDITION` before the order's status is touched.
- Streaming updates: `SubscribeToOrderUpdates` is a server-streaming RPC. On subscribe, the customer immediately receives the order's *current* status (so a client that subscribes after an update already happened isn't left waiting), then a fresh update is pushed via a per-subscriber queue. Queue every time `UpdateOrderStatus` or `CancelOrder` changes the order, until a terminal state (READY or CANCELLED) closes the stream from the server side.
- Exception handling: every one of the assignment's six required exceptional cases maps to a distinct, semantically appropriate gRPC status code - `NOT_FOUND` (unknown restaurant, unknown item, unknown order), `INVALID_ARGUMENT` (empty order, non-positive quantity), `FAILED_PRECONDITION` (invalid state transition, cancelling a non-PLACED order), and `PERMISSION_DENIED` (a restaurant attempting to update an order it does not own) - rather than a single generic error, so a client can programmatically distinguish and react to each failure mode.

5.2 .proto interface

`food_ordering.proto` defines `FoodOrderingService` with `ListRestaurants`, `PlaceOrder`, `GetOrderStatus`, `CancelOrder`, `ViewPendingOrders`, `UpdateOrderStatus`, and the server-streaming

SubscribeToOrderUpdates - matching every method named in the assignment's API list, plus CancelOrder and ViewPendingOrders as the natural server-side counterparts needed to implement the customer/restaurant CLI menus the assignment specifies.

5.3 Correctness verification

Automated test suite (test_food_ordering.py), 18/18 checks pass, covering:

- Restaurant listing returns the expected menu data.
- Order placement computes the correct total and starts at PLACED.
- The full happy-path lifecycle (ACCEPTED -> PREPARING -> READY), with a concurrently-subscribed customer receiving all four statuses (PLACED, ACCEPTED, PREPARING, READY) in order via the streaming RPC, without polling.
- 5 of the 6 required exceptional cases (non-existent restaurant, unavailable item, non-existent order, invalid transition, cross-restaurant update), each asserted against its specific expected gRPC status code (not just "any error").
- Cancellation rules cover the 6th required exceptional case: a freshly-PLACED order can be cancelled; an already-cancelled or already-ACCEPTED order cannot be (both correctly rejected with FAILED_PRECONDITION) - matching the spec's "customer attempts to cancel an order that has already been accepted or prepared" case.
- 20 simultaneous PlaceOrder calls from 20 threads, all receiving distinct order IDs with no lost or duplicated IDs - direct evidence the locking design in 5.1 is race-free under real concurrent load, not just in principle.

This suite was re-run fresh, from freshly regenerated .proto stubs, both locally and on a real RCE compute node, with identical results both times.

5.4 Performance study: order-placement throughput and latency vs. concurrent customers

50 orders per client thread, increasing client-thread count, measured locally:

Concurrent clients	Total orders	Elapsed (s)	Throughput (orders/s)	p50 latency (ms)	p95 latency (ms)
1	50	0.086	582.1	1.12	1.51
5	250	0.116	2149.8	2.12	2.95
10	500	0.215	2324.2	4.13	4.83
20	1000	0.428	2334.5	8.25	9.14

Observation: throughput jumps sharply from 1 to 5 concurrent clients (roughly 3.7x) because a single client thread spends most of its time idle, blocked on its own RPC round-trip, so a handful of concurrent clients fill that idle time almost for free; beyond 5 clients, throughput flattens out (2150 -> 2324 -> 2335 orders/s from 5 to 20 clients) while p50 latency keeps climbing (2.1ms -> 8.25ms) - i.e. additional concurrent clients past this point mostly wait in queue rather than add throughput. This saturation point lines up with server.py's ThreadPoolExecutor(max_workers=16): once concurrent in-flight requests

approach the size of the server's own worker pool, extra clients are throttled by request-thread availability rather than by the per-order locking design from 5.1 (test_food_ordering.py's 20-thread test confirms this queuing is safe, not a correctness problem - every request still completes with a unique order ID, just not instantaneously). Repeated runs of this benchmark show the same qualitative shape (sharp early gain, then a plateau) with the precise throughput numbers varying by up to ~30% run-to-run, as expected for a wall-clock microbenchmark sharing a machine with other processes; the trend, not the exact figures, is the reliable takeaway.

5.5 Verified on the actual RCE cluster (true cross-node run)

Beyond the automated test suite (re-run and passing on an RCE compute node), a genuine cross-node run was executed via rce_multinode_demo.sh: the server on one allocated node, a client probe on a second, physically separate node, connecting over the real cluster network rather than localhost:

```
Server node: node06
Client node: node07
[Probe] Restaurants: ['Pizza House', 'Burger Point']
[Probe] Order placed: 0101, total=250, status=PLACED
[Probe] UpdateOrderStatus ack: success=True message=Order 0101 -> ACCEPTED.
[Probe] Confirmed status via GetOrderStatus: ACCEPTED
[Probe] Cross-node gRPC communication verified OK.
```

5.6 Deliverables

Section3_gRPC/FoodOrdering/ - food_ordering.proto, server.py, customer.py, restaurant.py, the automated test suite, the concurrency benchmark script, and the multi-node RCE demo script, each with usage documented in the section README.

6. Execution compliance

Two hard constraints were stated repeatedly by course staff and are satisfied throughout every result reported above:

- Never run on the login/master node. Every single benchmark and correctness run in this report - local-only Python/C++ test runs excepted, which by definition run on whatever machine is developing the code - that touched the RCE cluster was launched via salloc, srun, or sbatch targeting allocated compute nodes, never executed directly on the login node's shell.
 - Multiple physical nodes, not multiple processes on one node, wherever the task calls for genuine distribution: the Section 1 scaling studies used 1/2/4/6 distinct SLURM-allocated nodes; the Section 2/3 gRPC cross-node demos explicitly placed the server, the ingesting/ordering client, and the dashboard/probe client on 2-3 different physical nodes and connected over the cluster's internal network address rather than localhost, so the RPCs genuinely crossed node boundaries.
-

7. Summary and conclusions

#	Section	Implementation	Correctness	Performance study	RCE execution
		n			
1	Sec 1 Q2	SSSP - iterative MapReduce	Matches PDF sample + independent Dijkstra up to V=1500	Node-count scaling (1/2/4/6) + size scaling (V=100/500/1500), with plots	Verified: real distributed runs at every configuration
2	Sec 2 Q1	Server Log Analytics - Hadoop Streaming (C++)	Matches HW2 sequential reference at N=10, 1k, 10k, 75k	Stage-by-stage timing at 3 sizes + quantitative MPI-vs-MapReduce comparison at HW2's own 4 dataset sizes, run on RCE	Attempted live twice; blocked by cluster-side outage (0 DataNodes; YARN ResourceManager unreachable), not by this implementation
3	Sec 2 Q2	Server Log Analytics - gRPC streaming	5/5 automated checks incl. concurrent ingest+query	Worker-count scaling + streaming-rate/granularity effect + query-latency-under-load (both required minimums covered)	Verified: real 3-node run
4	Sec 3	Food Ordering - gRPC	18/18 automated checks incl. 20-thread concurrency	Order-placement throughput/latency vs. concurrency	Verified: real cross-node run

Overall conclusion: all four required implementations are complete, independently correctness-verified against references that share no aggregation code with the implementation being tested, and benchmarked with real measurements (never estimated) covering both the dimensions the assignment explicitly asks for (node count, worker count, streaming granularity, concurrent query load) and the dimensions needed to explain **why** the numbers look the way they do (problem-size scaling for Section 1, per-stage timing breakdown for Section 2's Hadoop pipeline). The one implementation that could not be exercised end-to-end on the real cluster - Section 2's Hadoop job - was not left untested by omission: it was actually run against the live cluster twice, and the failure was root-caused to a cluster-side HDFS/YARN outage (documented with the exact error messages in Section 3.4) rather than to any defect in the submitted code, which is otherwise verified correct via the local pipeline simulation running the identical binaries.